

Metrics & Evaluation

Applied Machine Learning — Session 2, Chapter 3

The Metric Defines Success

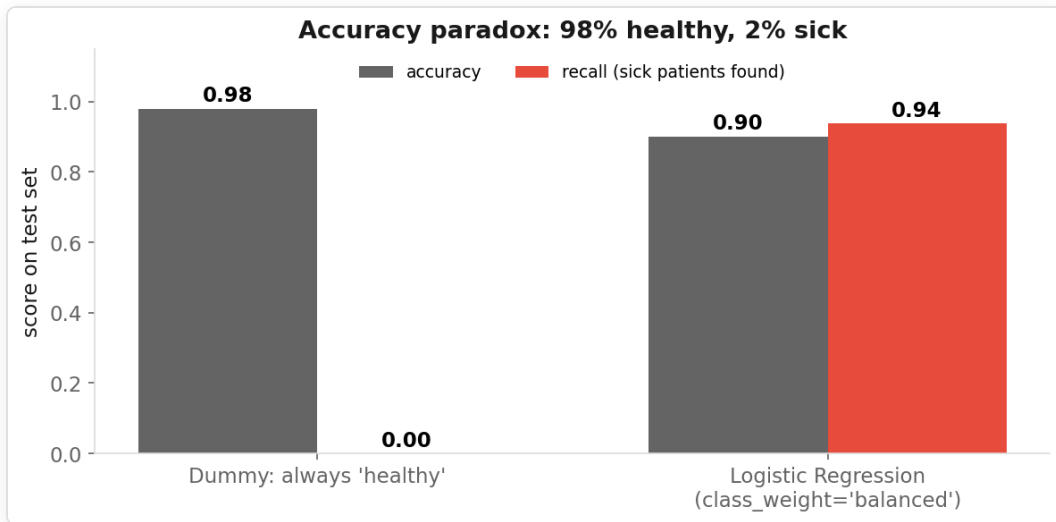
A model can only be as good as your definition of "good".

Screening example: 99 % of patients are healthy.

- Model that always says "healthy" → **99 % accuracy**
- ... and finds **0 %** of the sick patients

Choosing the wrong metric = optimizing for the wrong thing. Let's see it, not just say it →

The Accuracy Paradox — For Real



```
DummyClassifier(strategy='most_frequent')  
LogisticRegression(class_weight='balanced')
```

```
# 98 % accuracy, 0 % recall  
# 90 % accuracy, 94 % recall
```

Regression Metrics

Metric	Formula	Units	Best
MAE	$(1/n) \sum y_i - \hat{y}_i $	same as y	low
MSE	$(1/n) \sum (y_i - \hat{y}_i)^2$	y^2	low
RMSE	$\sqrt{\text{MSE}}$	same as y	low
R^2	$1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$	none	$\rightarrow 1$

```
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score
```

RMSE $\geq$ MAE always. A big gap = a few *large* errors. Use RMSE when big mistakes are what hurts.

R² — "How Much of the Variance Do We Explain?"

$R^2 = 1.0$ → perfect

$R^2 = 0.0$ → no better than always predicting the mean (= the DummyRegressor!)

$R^2 < 0.0$ → worse than the mean — something is broken

A "good" R^2 depends on the domain: finance 0.1 can be impressive · physics 0.99 is expected · house prices 0.6–0.8 typical.

Also look at the **residual plot** (demo Part A): errors should be random noise around 0 — any pattern = the model misses something.

The Confusion Matrix

	Predicted: healthy (0)	Predicted: cancer (1)
Actual: healthy (0)	TN healthy, predicted healthy ✓ correct	FP healthy, predicted cancer false alarm (Type I)
Actual: cancer (1)	FN cancer, predicted healthy MISSED CASE (Type II) △	TP cancer, predicted cancer ✓ correct

rows = truth, columns = prediction (sklearn convention) — positive class = 1 = the thing you look for

```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, display_labels=class_names)
tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()      # binary only, in this order
```

Worked Example: A Spam Filter

	predicted ham	predicted spam	
actual ham	TN = 90	FP = 5	← 5 good emails lost in the spam folder
actual spam	FN = 3	TP = 2	← only 2 of 5 spam caught

- **Accuracy** = $(90 + 2) / 100 = 0.92$ — sounds great
- **Precision** = $TP / (TP + FP) = 2 / 7 = 0.29$ — when it says "spam", it is wrong 5 times out of 7
- **Recall** = $TP / (TP + FN) = 2 / 5 = 0.40$ — it catches less than half of the spam

Different errors, different costs: a lost real email (FP) is worse than one spam in the inbox (FN) → for a spam filter, optimize **precision**.

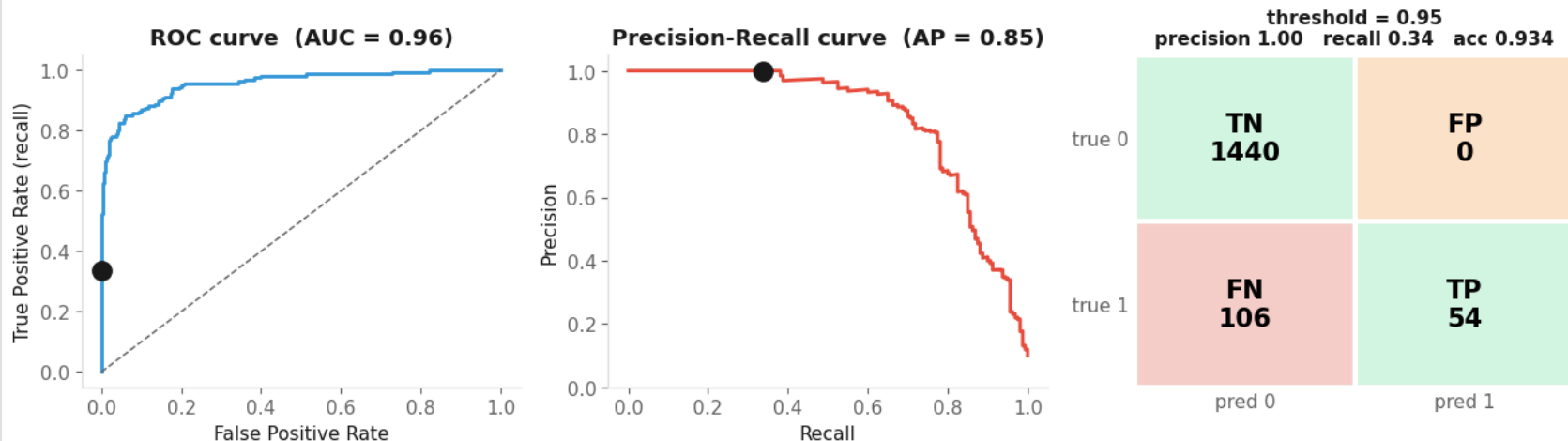
Precision, Recall, F1 — When to Use Which

Metric	Question it answers	Optimize when...	Example
Precision = $TP/(TP+FP)$	"When I say positive, am I right?"	FP is costly	spam filter
Recall = $TP/(TP+FN)$	"Did I find all positives?"	FN is costly	cancer screening
F1 = $2 \cdot P \cdot R / (P + R)$	balance of both (harmonic mean)	both matter, classes imbalanced	fraud, defect detection
Accuracy	share correct overall	classes balanced & errors equally bad	digit recognition

```
from sklearn.metrics import precision_score, recall_score, f1_score, classification_report
```

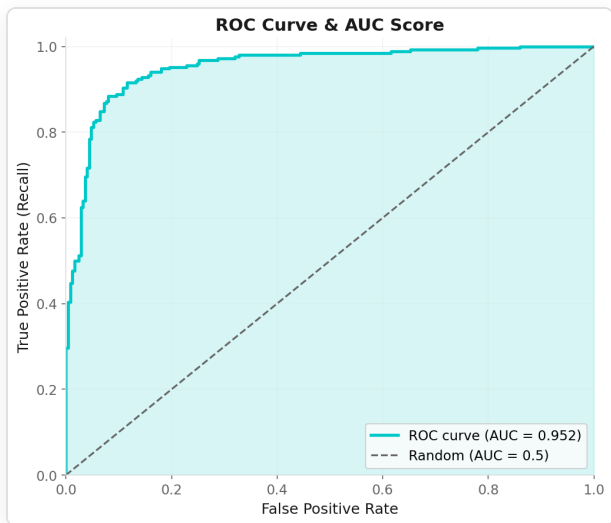

The Threshold Is Yours to Choose

Sliding the decision threshold: same model, very different trade-offs



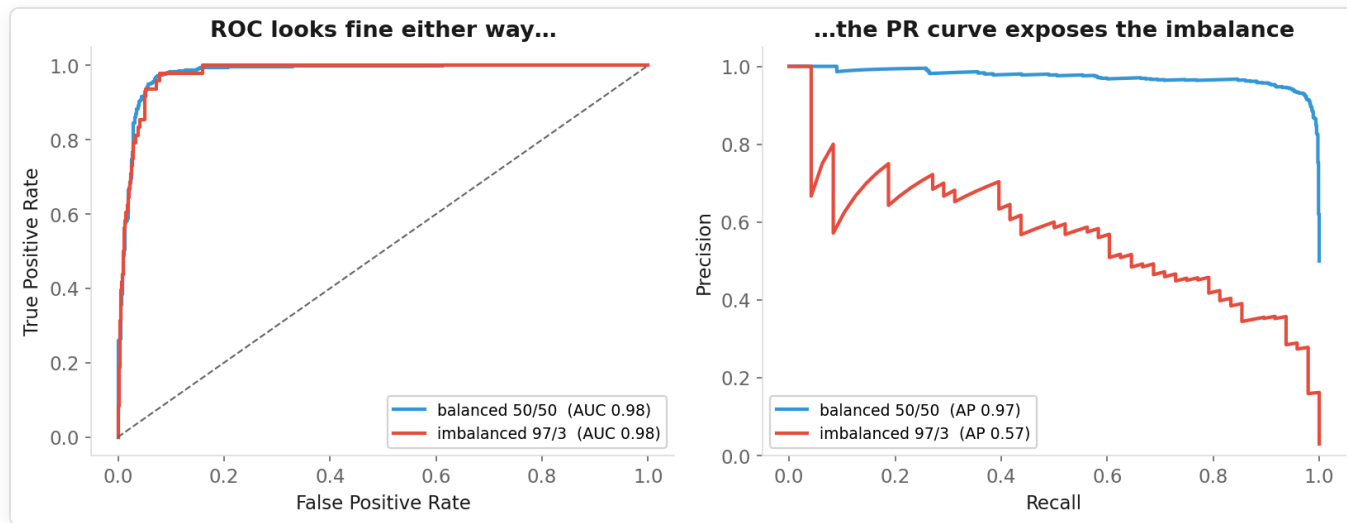
```
y_pred = (model.predict_proba(X_test)[: , 1] ≥ 0.3).astype(int) # instead of predict() = 0.5
```

ROC Curve & AUC



- **ROC:** True Positive Rate (recall) vs. False Positive Rate, one point per threshold
- **AUC** = area under it: 1.0 perfect · 0.5 random (diagonal) · *"P(random positive scores higher than random negative)"*
- Threshold-independent → compares the *ranking quality* of models

ROC vs. PR — and Handling Imbalance



- ROC's x-axis divides by the **big** class → looks fine even when positives are rare → use the **PR curve** then
- Toolbox for imbalance: `class_weight='balanced'` · lower threshold · `StratifiedKfold` · report **recall / F1 / PR-AUC**, not accuracy

Multi-Class: `classification_report` Averages

(illustrative example – 3 wine cultivars)

	precision	recall	f1-score	support	
class_0	1.00	0.92	0.96	12	
class_1	0.79	1.00	0.88	14	
class_2	1.00	0.90	0.95	10	
macro avg	0.93	0.94	0.93	36	← plain mean over classes (equal weight)
weighted avg	0.94	0.94	0.94	36	← weighted by support (big classes dominate)

For imbalanced multi-class problems report **macro F1**; `cross_val_score(..., scoring='f1_macro')`.

Cross-Validation with the *Right* Metric

```
from sklearn.model_selection import cross_val_score, StratifiedKFold

models = {
    'Logistic': make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),
    'KNN':      make_pipeline(StandardScaler(), KNeighborsClassifier(5)),
    'Forest':   RandomForestClassifier(100, random_state=42),
}

cv = StratifiedKFold(5, shuffle=True, random_state=42)
for name, m in models.items():
    s = cross_val_score(m, X, y, cv=cv, scoring='f1')    # or 'recall', 'roc_auc', 'f1_macro'
    print(f'{name}: {s.mean():.3f} ± {s.std():.3f}')
```

- The **pipeline** goes in → scaling re-fit per fold, no leakage • **stratified** folds keep the class ratio
- Pick `scoring` to match the cost of errors — then compare **mean ± std**

Quick Check

1. Screening model A: recall 0.98, precision 0.20. Model B: recall 0.80, precision 0.90. Which one for a first-line cancer screen? For deciding on surgery?
2. $AUC = 0.99$, but 3 of 42 cancers are missed at threshold 0.5. Contradiction?
3. A colleague reports "accuracy 0.97" on a fraud dataset with 1 % fraud. What do you ask?

Quick Check

1. Screening model A: recall 0.98, precision 0.20. Model B: recall 0.80, precision 0.90. Which one for a first-line cancer screen? For deciding on surgery?
 2. AUC = 0.99, but 3 of 42 cancers are missed at threshold 0.5. Contradiction?
 3. A colleague reports "accuracy 0.97" on a fraud dataset with 1 % fraud. What do you ask?
-
1. **A** for screening (missing a case is the expensive error; follow-up tests clean up the false alarms) — **B** when the action itself is costly/harmful
 2. No — AUC measures ranking across *all* thresholds; the deployed threshold decides FN. Lower it (Bonus A shows 3 → 1)
 3. "What is the recall / F1? What does the dummy score?" (0.99 accuracy — so 0.97 is *worse* than nothing)

Now: Exercises!

→ Open `03-exercises/ch06_metrics_exercises.ipynb` — models are pre-trained, you **evaluate** and **interpret**

Task	Content
A1	MAE / RMSE / R^2 for two diabetes models — better? by how much? (3 min)
B1	accuracy, precision, recall, F1 for the cancer model — which matters? (2 min)
B2	confusion matrix — how many cancers were missed? (2 min)
B3	ROC curve + AUC (3 min)
Bonus	threshold tuning · residual plot · dummy baseline

Positive class = 1 = malignant (already flipped in the setup cell).

Key Takeaways

- Accuracy lies on imbalanced data — always compare with a `DummyClassifier`
- Confusion matrix: rows = truth; positive class = what you look for — check the encoding!
- Precision (FP costly) vs. recall (FN costly) vs. F1 — pick the metric that matches the *cost of errors*
- The 0.5 threshold is a **decision**, not a law — slide it; ROC/PR curves show all thresholds at once
- Rare positives → PR curve, `class_weight`, stratified folds, F1/recall
- Cross-validate with a **pipeline** and the **right** `scoring`; report mean $\pm$ std

Next: Session 3

Unsupervised Learning

"What if we don't have labels? What can we still learn?"